Introduction to Python

Dennis Schroers, Ph.D., Institute of Finance and Statistics, University of Bonn

2026-10-01

This exercise sheet is a first introduction to **Python for Computational Statistics**. You do not need any previous programming experience. We begin by installing the software, opening a Jupyter notebook, and running a first command. Then we learn enough Python to work with a real dataset: daily Rhine River discharge at Cologne (Köln), obtained from the Global Runoff Data Centre.



Figure 1: The Rhine in Cologne.

Image credit: "Cologne (Köln) Rhine River view b.jpg". Original photograph by Neuwieser; edited by Georgfotoart. Source: Wikimedia Commons. License: CC BY-SA 2.0.

What you should be able to do afterwards: start Jupyter on your own; run and save Python code; work with arrays and tables; load and inspect a CSV file; calculate simple statistics; create plots; write a loop, a conditional, and a function; and generate reproducible pseudorandom numbers.

Download: Access the course's E-campus folder **datasets**. The course data file is `rhine_discharge_1816_2025.csv`. Download it and put it into the same folder as you Jupyter notebook that you create next after installation.

Getting started with Python

Our goal: Open a Jupyter notebook, run your first Python instructions, and save your work. You do not need any programming experience. We will use **Anaconda Navigator** throughout—no terminal commands are needed.

1. What do we need?

- **Python** is the programming language we will learn.
- **Anaconda Navigator** is an application that lets us start Python tools by clicking buttons.
- **Jupyter Notebook** is where we will write Python code. A notebook contains small boxes called **cells**, which we can run one at a time.

You only need to install **Anaconda Distribution**. It already includes Python; do **not** install Python separately.

2. Install Anaconda and open Navigator

1. Go to **https://www.anaconda.com/download**. Choose **Anaconda Distribution** (not Miniconda). If offered, you can select **Skip to download** instead of creating an account.
2. Download the installer for your computer:
   - **Windows:** Download the Windows installer (`.exe`). Open it and follow the instructions. The suggested settings are fine; choose **Just Me** if asked. Leave **Add Anaconda to my PATH** unchecked if that option appears.
   - **Mac with an Apple chip (M1, M2, M3, etc.):** Download the Apple Silicon installer (`.pkg`). Open it and follow the instructions.
3. Open **Anaconda Navigator**:
   - **Windows:** Open the Start menu and search for *Anaconda Navigator*.
   - **Mac:** Find *Anaconda Navigator* in Applications or search for it with Spotlight.

What you should see: A Navigator window with tiles for applications, including **Jupyter Notebook**. The appearance may vary slightly between versions.

Using an Intel Mac or Linux? Ask the teaching team for installation help; current Anaconda installation options differ for these systems.

3. Create your course folder and download the data

1. Open **File Explorer** (Windows) or **Finder** (Mac).
2. In **Documents**, create a folder named `computational-statistics`.
3. Log in to **E-campus** and open the course folder **datasets**.
4. Download `rhine_discharge_1816_2025.csv` and move it into `computational-statistics` if it was saved in Downloads.

Your folder should now contain:

```
Documents/
  computational-statistics/
    rhine_discharge_1816_2025.csv
```

Check: The filename ends in `.csv`, not `.csv.txt`. A CSV file is simply a table of data.

4. Open your first notebook

1. Return to **Anaconda Navigator** and select **Home**.
2. If there is a menu called **Applications on**, leave it set to **base (root)** (or **base**). This is the ready-to-use Python setup supplied with Anaconda; we do not need to create another one yet.
3. Find the **Jupyter Notebook** tile. Click **Launch**. If the button says **Install**, click **Install** first; then click **Launch**.
4. Jupyter should open in your web browser. You will see a list of files and folders. Open **Documents**, then **computational-statistics**. You should see your CSV file.
5. Create a notebook using **New → Notebook** or **New → Python 3** (the wording varies). If asked to choose a *kernel*, select **Python 3 (ipykernel)**.
6. Click the notebook title (often `Untitled`) and rename it **`intro_to_python`**. Jupyter saves it with the extension `.ipynb`.

What you should see: An empty notebook with a rectangular **code cell** where you can type.

5. Run your first Python code

1. Click inside the first code cell and enter:

   ```
   2 + 2
   ```

2. Press **Shift + Enter** to run the cell. You should see **4** below it.

3. In the next cell, enter:

   ```
   print("Hello, Python!")
   ```

4. Press **Shift + Enter** again. You should see:

```
Hello, Python!
```

5. Save with **Ctrl + S** (Windows) or **Cmd + S** (Mac).

Check: You have written, run, and saved your first Python notebook. Your course folder should now contain both `rhine_discharge_1816_2025.csv` and `intro_to_python.ipynb`.

6. How to continue next time

Open **Anaconda Navigator → Home → Jupyter Notebook → Launch**. In the browser, open **Documents → computational-statistics → intro_to_python.ipynb**. Run a cell with **Shift + Enter** and remember to save your notebook.

If something does not match these instructions: Check that you downloaded *Anaconda Distribution*, that Navigator is set to *base*, and that you opened the correct course folder. If you still get an error, show the message to the teaching team rather than installing another copy of Python.

1 First steps with Python

Run the code examples below in notebook cells, **in the order given**. A line beginning with `#` is a comment: Python ignores it. Some results are displayed automatically in a notebook, but `print(...)` works in scripts and notebooks.

Arithmetic, variables, and data types

```
print(2 + 2)
print(10 - 3)
print(4 * 7)
print(10 / 4)
print(2 ** 3)   # Exponentiation: 2 to the power 3
```

```
4
7
28
2.5
8
```

In Python, `=` assigns a value to a variable; `==` *checks* whether two values are equal. Python distinguishes between integers (`int`), decimal numbers (`float`), text (`str`), and logical values (`bool`).

```
x = 4
name = "Rhine"
is_large = x > 3
print(x, name, is_large)
print(type(x), type(name), type(is_large))
print(x == 4, x != 4)
```

```
4 Rhine True
<class 'int'> <class 'str'> <class 'bool'>
True False
```

Variable names are case-sensitive: `discharge` and `Discharge` are different. The symbol `^` is **not** exponentiation. Use `**` instead. If a command is unclear, try `help(print)` or the official Python tutorial.

Lists, NumPy arrays, and vectorized calculations

A Python **list** stores several values in order. For numerical calculations, **NumPy arrays** are particularly convenient. We import the package once and use its usual abbreviation `np`.

```
import numpy as np

numbers = [2, 7, 4, 1]        # A Python list
```

```
y = np.array(numbers)        # A NumPy array
print("List:", numbers)
print("Array:", y)
print("Array shape:", y.shape)
```

```
List: [2, 7, 4, 1]
Array: [2 7 4 1]
Array shape: (4,)
```

For arrays, multiplication and mathematical functions act **element by element** unless a matrix operation is specified. This is called *vectorization*.

```
print("Multiply by 4:", 4 * y)
print("Squares:", y ** 2)
print("Logarithms:", np.log(y))
print("Mean:", np.mean(y))
print("Centered:", y - np.mean(y))
print("Standardized:", (y - np.mean(y)) / np.std(y, ddof=1))
```

```
Multiply by 4: [ 8 28 16  4]
Squares: [ 4 49 16  1]
Logarithms: [0.69314718 1.94591015 1.38629436 0.        ]
Mean: 3.5
Centered: [-1.5  3.5  0.5 -2.5]
Standardized: [-0.56694671  1.32287566  0.18898224 -0.94491118]
```

The argument `ddof=1` chooses the sample standard deviation, with divisor $n-1$ rather than n.

Comparisons and conditions (`if`, `elif`, `else`)

Conditions make a program choose between alternatives. Comparisons include `>`, `<`, `>=`, `<=`, `==` (equal) and `!=` (not equal). A condition has the value `True` or `False`. **Indentation** determines which lines belong to a conditional block.

```
water_level = 5
if water_level > 5:
    print("Above the threshold")
elif water_level == 5:
    print("Exactly at the threshold")
else:
    print("Below the threshold")
```

```
Exactly at the threshold
```

When combining simple conditions, use `and`, `or`, and `not`. With NumPy arrays or pandas columns, use the elementwise operators `&` (and) and `|` (or), with parentheses around each comparison. We will use these when filtering data.

Loops (`for`) and sequences

A **loop** repeats the same instructions for a collection of values. `range(4)` produces the integers 0, 1, 2, 3. Python starts counting array positions at **zero**.

```
y_sq = np.zeros(len(y))
for i in range(len(y)):
    y_sq[i] = y[i] ** 2
print("Squares computed in a loop:", y_sq)
```

```
Squares computed in a loop: [ 4. 49. 16.  1.]
```

A loop is sometimes easier to understand or adapt than a single vectorized expression. When possible, NumPy vectorization is often more concise.

```
print(np.arange(1, 11))      # 1, 2, ..., 10; upper bound excluded
print(np.arange(-3, 4))      # -3, -2, ..., 3
print(np.arange(1, 20, 3))   # Step size 3
```

```
[ 1  2  3  4  5  6  7  8  9 10]
[-3 -2 -1  0  1  2  3]
[ 1  4  7 10 13 16 19]
```

Exercise 1. Create an array with the values `1, 2, 3, ..., 10`. Write a `for` loop that computes the natural logarithm of every element and stores the results in a new array. Print the array and its mean. Then compute the same values directly with `np.log(...)` and compare the results.

Solution to Exercise 1

```
y_ex1 = np.arange(1, 11)
y_log = np.zeros(len(y_ex1))
for i in range(len(y_ex1)):
    y_log[i] = np.log(y_ex1[i])
print("Logarithms:", y_log)
print("Mean logarithm:", np.mean(y_log))
print("Same result as NumPy?", np.allclose(y_log, np.log(y_ex1)))
```

```
Logarithms: [0.         0.69314718 1.09861229 1.38629436 1.60943791 1.79175947
 1.94591015 2.07944154 2.19722458 2.30258509]
Mean logarithm: 1.5104412573075514
Same result as NumPy? True
```

Matrices, indexing, and Boolean masks

A matrix is a **two-dimensional array**. NumPy's `reshape` takes a sequence and arranges it into rows and columns. Here is a 4×4 matrix:

```
A = np.arange(1, 17).reshape((4, 4))
print(A)
print("Dimensions:", A.shape)
print("First row:", A[0, :])
print("First column:", A[:, 0])
print("Bottom-right element:", A[3, 3])
print("First two elements of y:", y[0:2])
```

```
[[ 1  2  3  4]
 [ 5  6  7  8]
 [ 9 10 11 12]
 [13 14 15 16]]
Dimensions: (4, 4)
First row: [1 2 3 4]
First column: [ 1  5  9 13]
Bottom-right element: 16
First two elements of y: [2 7]
```

In the slice `0:2`, the final position **2 is excluded**: the result contains positions 0 and 1. The symbol `:` by itself means *all* entries along that axis. To multiply matrices use `@`; ordinary `*` multiplies their corresponding entries. To compute an inner product of vectors use `np.dot` or `@`.

```
print("Inner product:", y @ y)
print("Matrix product:", A @ A)
mask = y > 3
print("Condition for each element:", mask)
print("Elements greater than 3:", y[mask])
```

```
Inner product: 70
Matrix product: [[ 90 100 110 120]
 [202 228 254 280]
```

```
 [314 356 398 440]
 [426 484 542 600]]
Condition for each element: [False  True  True False]
Elements greater than 3: [7 4]
```

Lists, dictionaries, and DataFrames

A **dictionary** attaches names (called *keys*) to values. A **DataFrame**, from the `pandas` package, is a table whose columns have names. A pandas **Series** is a single column of values. These will be central when handling datasets.

```
student_example = {"group": "A", "measurements": [3.1, 2.8, 4.0]}
print(student_example["group"])

import pandas as pd
example_table = pd.DataFrame({
    "Day": ["Monday", "Tuesday", "Wednesday"],
    "Discharge": [1900, 2150, 1800],
})
print(example_table)
print(example_table["Discharge"])
print("Column types:\n", example_table.dtypes)
```

```
A
         Day  Discharge
0     Monday       1900
1    Tuesday       2150
2  Wednesday       1800
0    1900
1    2150
2    1800
Name: Discharge, dtype: int64
Column types:
 Day          object
Discharge     int64
dtype: object
```

2 Working with the Rhine discharge data

Our dataset contains **daily mean discharge** at **Köln (Cologne)** on the Rhine River. The measurements run from **1 November 1816 through 31 December 2025**, with **76,397 daily rows**. Discharge is the volume of water passing a river cross-section per unit time, measured here in **cubic metres per second (m^3/s)**. These are observations of *discharge*, not water levels.

The source is the **Global Runoff Data Centre (GRDC), station 6335060**; the original data owner is the German Federal Waterways Administration (WSV). The course provides a simplified two-column CSV. The CSV has the column `Date` followed by `Discharge`. The original source denotes unavailable measurements with the number `-999`; our course copy contains no such entries, but the code below handles that convention defensively.

Before running the next cell: download `rhine_discharge_1816_2025.csv` from **E-campus > datasets** and place it in the same `computational-statistics` folder as your notebook.

Load and inspect a CSV

```
from pathlib import Path

print("Current working folder:", Path.cwd())
csv_path = Path("rhine_discharge_1816_2025.csv")
if not csv_path.is_file():
```

```
    raise FileNotFoundError(
        "The CSV was not found. Download rhine_discharge_1816_2025.csv "
        "from E-campus > datasets and save it in the folder printed above."
    )

rhine = pd.read_csv(csv_path, parse_dates=["Date"])
rhine["Discharge"] = pd.to_numeric(rhine["Discharge"], errors="coerce")
rhine.loc[rhine["Discharge"] == -999, "Discharge"] = np.nan
print(rhine.head())
```

```
Current working folder: /Users/eunsilkang/Desktop/Computer_Aided_Statistics/intro_to_python
        Date  Discharge
0 1816-11-01     1410.0
1 1816-11-02     1410.0
2 1816-11-03     1410.0
3 1816-11-04     1410.0
4 1816-11-05     1540.0
```

Here `pd.read_csv(...)` reads the file into a DataFrame. `parse_dates` makes `Date` a date column rather than plain text, so that we can later choose a year. `np.nan` is NumPy's marker for missing numerical values.

Always inspect newly imported data **before** calculating statistics:

```
print("Rows, columns:", rhine.shape)
print("Column names:", list(rhine.columns))
print("First and last dates:", rhine["Date"].min(), rhine["Date"].max())
print("Missing values in each column:\n", rhine.isna().sum())
print("Last three rows:\n", rhine.tail(3))
print("Column types:\n", rhine.dtypes)
```

```
Rows, columns: (76397, 2)
Column names: ['Date', 'Discharge']
First and last dates: 1816-11-01 00:00:00 2025-12-31 00:00:00
Missing values in each column:
 Date         0
Discharge    0
dtype: int64
Last three rows:
             Date  Discharge
76394 2025-12-29     1140.0
76395 2025-12-30     1110.0
76396 2025-12-31     1080.0
Column types:
 Date         datetime64[ns]
Discharge           float64
dtype: object
```

If you get `FileNotFoundError`, look at the folder printed by `Path.cwd()`: it is the **working directory**. Move your CSV there, or supply its complete path with `Path(...)`. On Windows, for example, `Path(r"C:\Users\yourname\Documents\computational-statistics\rhine_discharge_1816_2025.csv")` uses a *raw string*; you must replace `yourname` with your real folder name.

Select columns and filter observations

`rhine["Discharge"]` extracts one column as a Series. A comparison on a whole column generates one Boolean (`True` or `False`) per row. With `.loc[...]`, we can keep exactly the rows satisfying the condition.

```
discharge = rhine["Discharge"]
rhine_2024 = rhine.loc[rhine["Date"].dt.year == 2024].copy()
print("Days in 2024:", len(rhine_2024))
print(rhine_2024.head(3))
```

```
Days in 2024: 366
            Date  Discharge
75666 2024-01-01     3880.0
75667 2024-01-02     3650.0
75668 2024-01-03     4150.0
```

The `.dt.year` expression extracts the year from a pandas date column. The `.copy()` makes a separate table so that later modifications cannot change the original by accident. Conditions can also be combined:

```
# Dates in 2024 for which discharge exceeded 5000 m³/s
high_flow_2024 = rhine.loc[
    (rhine["Date"].dt.year == 2024) & (rhine["Discharge"] > 5000)
]
print("Days above 5000 m³/s in 2024:", len(high_flow_2024))
print(high_flow_2024.head())
```

```
Days above 5000 m³/s in 2024: 8
            Date  Discharge
75669 2024-01-04     5730.0
75670 2024-01-05     6640.0
75671 2024-01-06     6750.0
75672 2024-01-07     6030.0
75673 2024-01-08     5200.0
```

Try changing `5000` to another threshold, or select a different year. Remember that `.loc[rows, columns]` can also select particular columns, for example `rhine.loc[rhine["Date"].dt.year == 2024, "Discharge"]`.

Descriptive statistics

The *sample mean* is $\bar{x} = \frac{1}{n}\sum_{i=1}^{n} x_i$. The *sample variance* and *sample standard deviation* are

$$s^2 = \frac{1}{n-1}\sum_{i=1}^{n}(x_i - \bar{x})^2, \qquad s = \sqrt{s^2}.$$

The functions below are applied to the full 1816–2025 series. `ddof=1` uses the denominator $n-1$; `ddof=0` would use n.

```
print("Mean (m³/s):", discharge.mean())
print("Sample variance:", discharge.var(ddof=1))
print("Sample standard deviation (m³/s):", discharge.std(ddof=1))
print("Minimum (m³/s):", discharge.min())
print("Maximum (m³/s):", discharge.max())
print("Quantiles (m³/s):\n", discharge.quantile([0, 0.25, 0.5, 0.75, 1]))
print("Square-root check:", np.isclose(
    np.sqrt(discharge.var(ddof=1)), discharge.std(ddof=1)))
```

```
Mean (m³/s): 2079.508279120908
Sample variance: 1081007.4021416523
Sample standard deviation (m³/s): 1039.7150581489393
Minimum (m³/s): 401.0
Maximum (m³/s): 10900.0
Quantiles (m³/s):
 0.00      401.0
0.25     1400.0
0.50     1840.0
0.75     2450.0
1.00    10900.0
Name: Discharge, dtype: float64
Square-root check: True
```

These summary statistics describe the observed values, but they do not show *when* the river was high or low. Time-series plots address that question. Also remember that these are consecutive daily measurements, not

independent random draws; this distinction matters for later statistical modelling.

Missing values

Real datasets may contain missing observations. In this CSV, none are missing. To practise, we make a **copy** of the discharge Series and deliberately replace five values with `np.nan`, using a fixed random seed to select the positions reproducibly.

```
rng_missing = np.random.default_rng(123)
q_missing = discharge.copy()
missing_positions = rng_missing.choice(len(q_missing), size=5, replace=False)
q_missing.iloc[missing_positions] = np.nan
print("Missing in original:", discharge.isna().sum())
print("Missing in modified copy:", q_missing.isna().sum())
```

```
Missing in original: 0
Missing in modified copy: 5
```

The pandas mean skips missing values by default. By contrast, an ordinary NumPy mean of an array containing `np.nan` returns `nan`. The `np.nanmean(...)` function explicitly excludes missing observations.

```
print("NumPy mean:", np.mean(q_missing.to_numpy()))
print("pandas mean:", q_missing.mean())
print("NumPy mean ignoring NaNs:", np.nanmean(q_missing.to_numpy()))
```

```
NumPy mean: nan
pandas mean: 2079.4405696931617
NumPy mean ignoring NaNs: 2079.4405696931617
```

3 Plotting data

We use **Matplotlib**, imported as `plt`. First we plot the daily measurements for **2024**. This is the natural starting point for data recorded over time.

Time-series plot

```
import matplotlib.pyplot as plt

fig, ax = plt.subplots()
ax.plot(rhine_2024["Date"], rhine_2024["Discharge"], linewidth=1)
ax.set(title="Rhine discharge at Köln, 2024",
       xlabel="Date", ylabel="Daily mean discharge (m³/s)")
fig.autofmt_xdate()
plt.show()
```

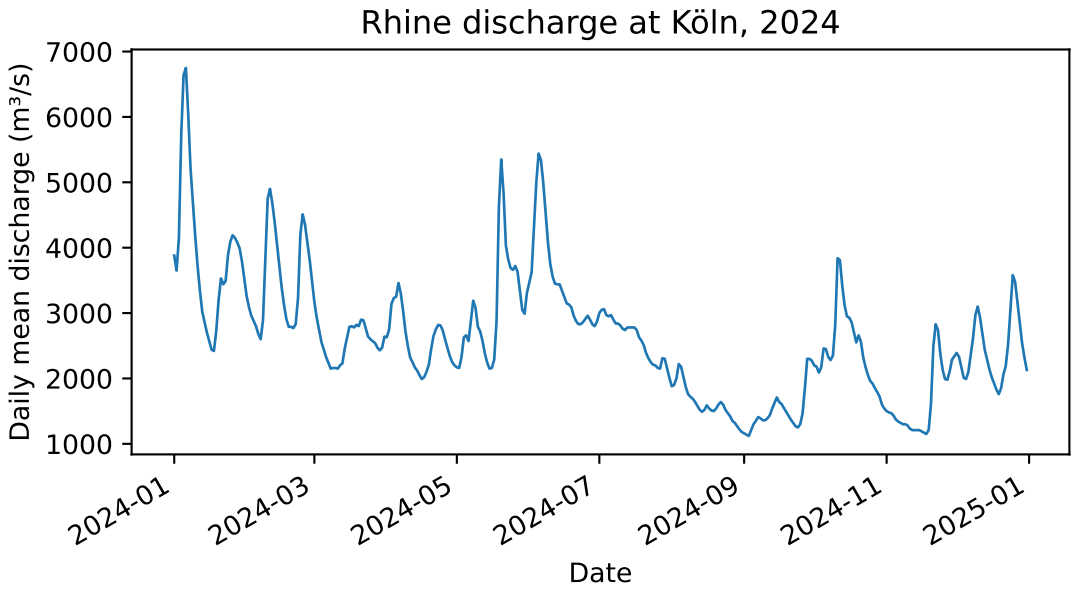


Figure 2: Daily mean Rhine discharge at Köln during 2024.

The horizontal axis shows **when** observations occurred, so the chart makes changes and peaks over time visible. The next two plots instead summarize the **distribution** of values without displaying their chronological order.

Histogram and a reference normal density

A histogram divides the discharge range into intervals (*bins*) and shows how many observations fall within each interval. Setting `density=True` puts the histogram on a density scale. We also draw a normal density with the same mean and standard deviation as the 2024 measurements.

```
from scipy.stats import norm

q_2024 = rhine_2024["Discharge"].dropna()
mu_2024 = q_2024.mean()
sd_2024 = q_2024.std(ddof=1)
grid = np.linspace(max(0, q_2024.min() - 300), q_2024.max() + 300, 250)

fig, ax = plt.subplots()
ax.hist(q_2024, bins=25, density=True, color="lightgray",
        edgecolor="white", label="Observed discharge")
ax.plot(grid, norm.pdf(grid, loc=mu_2024, scale=sd_2024),
        color="black", linewidth=1.4, label="Normal reference")
ax.set(xlabel="Daily mean discharge (m³/s)", ylabel="Density")
ax.legend()
plt.show()
```

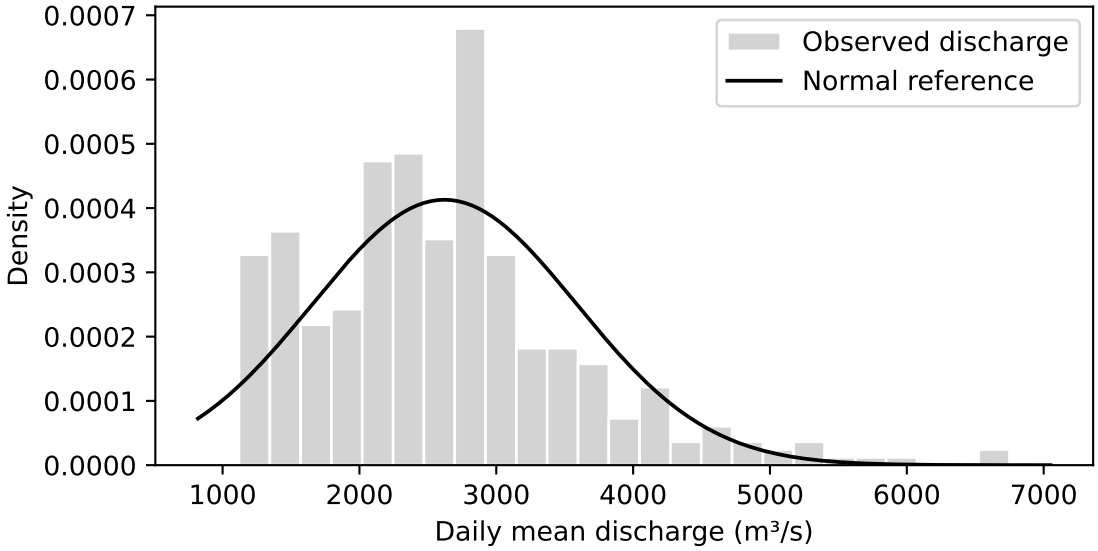


Figure 3: Histogram of daily discharge in 2024 with a fitted normal reference density.

The normal curve is only a **reference**. River discharge is nonnegative, may be skewed, and often exhibits dependence between successive days. A fitted normal curve does not establish that the data are normally distributed.

Exercise 2. Make a **horizontal boxplot** of discharge in 2024, and add the individual daily values as points. Label the horizontal axis in m^3/s. What do the boxplot and the histogram show that the time-series plot does not?

Solution to Exercise 2

```
fig, ax = plt.subplots(figsize=(6.3, 2.4))
ax.boxplot(q_2024, vert=False, positions=[1], widths=0.35)
ax.scatter(q_2024, np.full(len(q_2024), 0.70),
           facecolors="none", edgecolors="black", s=12, linewidths=0.5)
ax.set(xlabel="Daily mean discharge (m³/s)", yticks=[1], yticklabels=[""])
ax.set_ylim(0.45, 1.35)
plt.show()
```

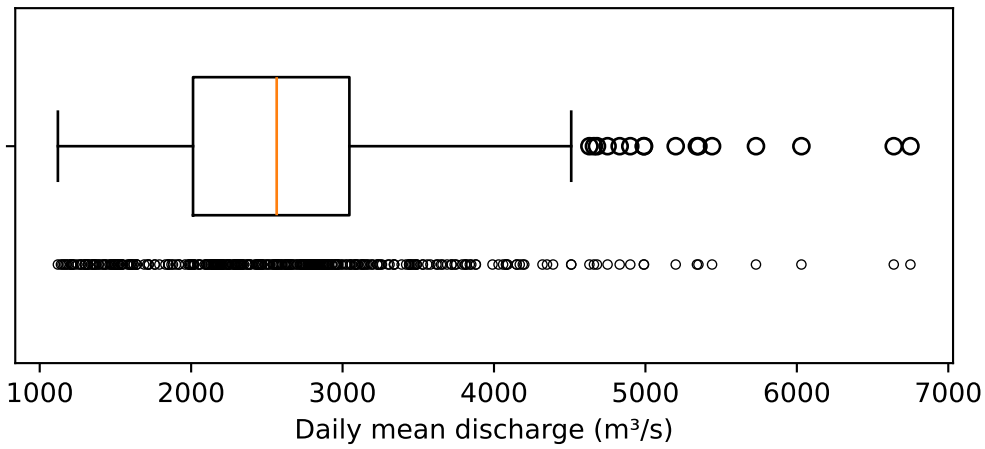


Figure 4: Boxplot and individual daily discharge observations from 2024.

The histogram and boxplot show how widely the daily discharges vary and whether large observations occur. Only the time-series plot shows *when* they occur.

4 Programming with Python

Define your own functions

A **function** packages instructions under a name. We define functions with `def`, place the calculations in an indented block, and use `return` to pass back the result.

```
def my_mean(values):
    n = len(values)
    return sum(values) / n

x = q_2024.to_numpy()
print("Our mean:", my_mean(x))
print("NumPy mean:", np.mean(x))
```

```
Our mean: 2621.3387978142077
NumPy mean: 2621.3387978142077
```

The names `values` and `n` inside the function describe intermediate objects. You can call the same function with different data. Here we assume that the input is nonempty and has no missing values; production code would need more checks.

Exercise 3. Write a function `my_var(x, unbiased=True, na_rm=False)` for the empirical variance. When `unbiased=True`, use divisor $n-1$; otherwise use n. When `na_rm=True`, discard missing values before calculating. Compare with NumPy or pandas.

Solution to Exercise 3

```
def my_var(x, unbiased=True, na_rm=False):
    values = np.asarray(x, dtype=float)
    if na_rm:
        values = values[~np.isnan(values)]
    n = len(values)
    if n == 0 or (unbiased and n < 2):
        return np.nan
    if np.isnan(values).any():
        return np.nan
    average = np.mean(values)
    denominator = n - 1 if unbiased else n
    return float(np.sum((values - average) ** 2) / denominator)

print("Sample variance:", my_var(q_2024))
print("pandas check:", q_2024.var(ddof=1))
print("Variance with divisor n:", my_var(q_2024, unbiased=False))
print("Missing values retained:", my_var(q_missing))
print("Missing values removed:", my_var(q_missing, na_rm=True))
```

```
Sample variance: 933269.7095590988
pandas check: 933269.7095590993
Variance with divisor n: 930719.7923198664
Missing values retained: nan
Missing values removed: 1080712.3608564674
```

The `if` statements make the function handle empty inputs and missing values. The function illustrates programming logic; for routine analysis, established library functions are usually preferable.

Pseudorandom numbers and reproducibility

Much of Computational Statistics uses **simulation**. A computer generates pseudorandom numbers using an algorithm. A **seed** fixes its initial state, allowing someone else to reproduce the same draws and results.

Here we simulate 1,000 independent observations from a standard normal distribution, $N(0,1)$. For each draw the theoretical mean is 0 and standard deviation is 1, but the empirical values will not be exactly equal to those numbers.

```
rng = np.random.default_rng(2026)
simulated = rng.normal(loc=0, scale=1, size=1000)
print("First five simulated values:", simulated[:5])
print("Simulated sample mean:", np.mean(simulated))
print("Simulated sample SD:", np.std(simulated, ddof=1))
```

```
First five simulated values: [-0.79312248  0.24057128 -1.89632635  1.39577171  0.63829474]
Simulated sample mean: 0.021067219889700724
Simulated sample SD: 1.0309712211114948
```

Running `rng.normal(...)` **again** advances the generator and gives different values. Creating a *new* generator with the **same seed** reproduces the original sequence:

```
rng_again = np.random.default_rng(2026)
again = rng_again.normal(loc=0, scale=1, size=1000)
print("Exactly the same simulated values?", np.array_equal(simulated, again))
```

```
Exactly the same simulated values? True
```

This is why reporting a seed can be important for reproducible analysis. Later in the course you will use simulation for questions that cannot be answered by simple formulas alone.

5 Putting it together: a first independent analysis

Exercise 4. Use the Rhine CSV to complete the following tasks.

1. Select daily measurements from **2020 through 2024**, inclusive.
2. Calculate the **average daily discharge in each of the five years**.
3. Identify which of these years had the **highest annual mean discharge**.
4. Create a plot of the five annual means with meaningful axis labels.

Hint: `rhine["Date"].dt.year` extracts the year. The pandas method `.groupby("Year")["Discharge"].mean()` computes one mean per year. You can use `ax.plot(...)` or `ax.bar(...)` to show the five annual averages.

Solution to Exercise 4

```
recent = rhine.loc[rhine["Date"].dt.year.between(2020, 2024)].copy()
recent["Year"] = recent["Date"].dt.year
annual_means = recent.groupby("Year")["Discharge"].mean()
print("Annual means (m³/s):\n", annual_means)
print("Highest annual mean in:", annual_means.idxmax())

fig, ax = plt.subplots()
ax.plot(annual_means.index, annual_means.values, marker="o")
ax.set(xlabel="Year", ylabel="Annual mean discharge (m³/s)",
       title="Rhine discharge at Köln: annual means")
ax.set_xticks(annual_means.index)
plt.show()
```

```
Annual means (m³/s):
 Year
2020    1766.057377
2021    2131.271233
2022    1586.786301
2023    2182.183562
2024    2621.338798
Name: Discharge, dtype: float64
Highest annual mean in: 2024
```

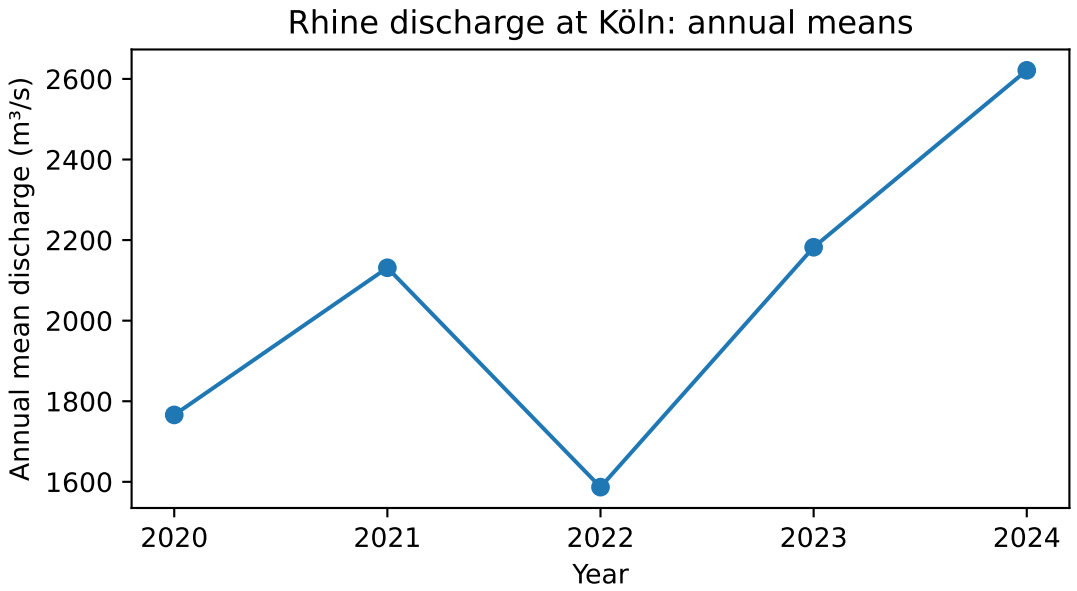


Figure 5: Mean daily Rhine discharge at Köln for each year from 2020 to 2024.

The average of the daily values is the **annual mean discharge** for each year. It should not be confused with annual total water volume. Notice also that a single-year average summarizes that year but does not show the daily variation within it.

Common problems and how to solve them

If something does not work, read the **last line of the error message** first. It often names the type of problem. These are particularly common:

What you see	Likely cause	What to try
`NameError: name 'rhine' is not defined`	The cell that created `rhine` was not run, or the kernel was restarted	Run the earlier cells from top to bottom
`ModuleNotFoundError: No module named 'pandas'`	Package is missing in the active kernel/environment	Select the `compstats-python` kernel; check package installation in the terminal
`FileNotFoundError`	CSV is not in the folder where Python is looking	Run `print(Path.cwd())`; move the file or correct its path
`SyntaxError`	Invalid Python syntax	Check quotation marks, commas, brackets and colons
`IndentationError`	Lines inside `if`, `for` or `def` are not indented consistently	Use four spaces for each block level
A variable has an unexpected value	Notebook cells were executed out of order	Restart the kernel and run all cells in order
A figure is not visible	Plot cell was not executed, or output was cleared	Run the cell again, including `plt.show()`

A simple debugging technique is to insert `print(...)` statements to inspect intermediate results, shapes, and types:

```
print("Type:", type(rhine))
print("Shape:", rhine.shape)
print("Missing discharge observations:", rhine["Discharge"].isna().sum())
```

```
Type: <class 'pandas.core.frame.DataFrame'>
```

```
Shape: (76397, 2)
Missing discharge observations: 0
```

Do not worry about memorizing every command. Instead, practise reading the code, making small changes, rerunning cells, and checking whether the resulting numbers and plots make sense.